

## ASSIGNMENT (3)

---

### Part 1: Multiple Choice (MCQ)

1. **Change tolerance normally involves:** Answer: b) Incremental development
  2. **A prototype is usually used in:** Answer: a) Requirements engineering
  3. **Throw-away prototypes should be discarded mainly because:** Answer: b) They cannot meet non-functional requirements
  4. **In incremental delivery:** Answer: b) Requirements for later increments may continue to evolve
  5. **Which model represents the process as cycles rather than phases?** Answer: c) Spiral model
  6. **In the Spiral Model, risk assessment happens:** Answer: c) Continuously in every loop
  7. **RUP phases include all of the following except:** Answer: c) Deployment
  8. **Which RUP practice emphasizes the use of UML diagrams?** Answer: d) Visually model software
  9. **A key advantage of incremental delivery is:** Answer: b) Delivery of customer value with each increment
  10. **One major problem with incremental delivery is:** Answer: b) Hard to identify basic facilities that all increments need
- 

### Part 3: Essay Questions

#### 1. Role of Prototyping in Software Development

Prototyping is a technique where a preliminary version of the software system is built to understand and refine requirements. Its role includes:

- **Requirements Gathering:** Stakeholders can interact with a prototype to clarify functional requirements, leading to fewer misunderstandings and improved requirement accuracy.
- **Design:** Prototypes help designers experiment with UI layouts, workflows, and system interactions, enabling better design decisions before final implementation.

- **Testing:** Early versions of the system allow for initial testing, especially for usability and functionality, helping identify flaws early.

**Key Contribution:** Prototyping reduces the risk of building the wrong system and ensures higher user satisfaction.

---

## 2. Incremental Delivery Model

The incremental delivery model divides the system into small, manageable modules delivered in increments. Each increment adds functionality until the complete system is delivered.

**Advantages:** - Early delivery of partial system benefits to users. - Changes can be incorporated in later increments. - Reduced risk of complete project failure due to iterative feedback.

**Challenges:** - Difficult to predict the total project timeline. - Identifying core features common to all increments can be hard. - Integration between increments must be carefully managed.

---

## 3. Boehm's Spiral Model

**Main Phases of the Spiral Model:** 1. **Planning:** Identify objectives, alternatives, and constraints. 2. **Risk Analysis:** Assess technical, schedule, and cost risks. 3. **Engineering:** Develop and verify the next iteration of the product. 4. **Evaluation:** Review progress with stakeholders and gather feedback.

**Why Risk-Driven:** - Each iteration explicitly identifies and mitigates risks before proceeding, reducing the likelihood of project failure. - Focus on risk management makes it suitable for large, complex, and high-risk projects.

---

## 4. Rational Unified Process (RUP)

**RUP Overview:** RUP is a software development process framework that provides structured approaches to building software. It is iterative and use-case driven, focusing on risk mitigation and incremental development.

**Perspectives of RUP:**

### 1. Dynamic Perspective (Phases):

- **Inception:** Define scope, goals, and feasibility.
- **Elaboration:** Refine architecture, mitigate risks, and finalize requirements.

- **Construction:** Build the product incrementally and iteratively.
- **Transition:** Deploy the system to users, gather feedback, and correct defects.

2. **Static Perspective (Artifacts):**

- Documents, models, code, and other outputs of the process.
- Includes use-case models, design models, test plans, and deployment models.

3. **Practice Perspective (Best Practices):**

- **Develop iteratively:** Avoid building everything at once.
- **Manage requirements:** Keep track of requirements systematically.
- **Use component-based architecture:** Promote modularity and reuse.
- **Visually model software:** Use UML for clarity.
- **Continuously verify quality:** Testing throughout development.
- **Control changes:** Manage modifications in requirements and code.